

**CUỘC THI KHOA HỌC, KỸ THUẬT DÀNH CHO HỌC SINH TRUNG HỌC
CẤP TRƯỜNG NĂM HỌC 2026 – 2027**

BẢN BÁO CÁO TÓM TẮT KẾT QUẢ NGHIÊN CỨU

Tên đề tài:

**ỨNG DỤNG CƠ CHẾ GIẢI PHÓNG DỮ LIỆU ĐỆM
TRONG PHÁT TRIỂN AI**

Lĩnh vực: Phần mềm hệ thống

Hà Nội, tháng 9 năm 2026

MỤC LỤC

PHẦN I:	Tổng quan dự án	4
I.1.	Lý do chọn dự án	4
I.2.	Câu hỏi nghiên cứu	4
I.3.	Mục tiêu nghiên cứu	4
I.4.	Tính mới nghiên cứu	5
PHẦN II:	Cơ sở lý thuyết	6
II.1.	Ngữ nghĩa đọc phá hủy	6
PHẦN III:	Thiết kế và phương pháp	7
III.1.	Thiết kế cú pháp	7
III.2.	Quy trình xử lý	7
III.3.	Cơ chế giải phóng khi tiêu thụ và runtime cuBLAS	8
PHẦN IV:	Quy trình, chế tạo và kiểm thử	9
IV.1.	Quy trình thực hiện	9
IV.2.	Kiểm thử	10
IV.3.	Đánh giá hiệu năng	10
IV.4.	Mối quan hệ kỳ vọng giữa ngôn ngữ và hiệu năng nhân ma trận	11
IV.5.	Minh chứng và khả năng tái lập	12
PHẦN V:	Kết luận	12
V.1.	Tiến độ	12
V.2.	Hướng phát triển	12
V.3.	Ý nghĩa	13
TÀI LIỆU THAM KHẢO		13

DANH MỤC HÌNH VẼ, SƠ ĐỒ

Hình 1.	Quy trình biên dịch của ngôn ngữ. Các khối tô xám biểu thị những bước liên quan đến ngữ nghĩa đọc phá hủy.	8
---------	--	---

DANH MỤC BẢNG BIỂU

Bảng 1.	Các lựa chọn hiện thực cơ chế trong thiết kế.	9
Bảng 2.	Các tệp mã nguồn chính	9
Bảng 3.	Kết quả kiểm tra tính đúng đắn	10

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tên đầy đủ	Ý nghĩa
AI	Artificial Intelligence	Trí tuệ nhân tạo
AOT	Ahead-Of-Time (compilation)	Biên dịch trước khi chạy
AST	Abstract Syntax Tree	Cây cú pháp trừu tượng
cuBLAS	CUDA Basic Linear Algebra Subprograms	Thư viện đại số tuyến tính cơ bản trên CUDA
CUDA	Compute Unified Device Architecture	Nền tảng tính toán song song trên GPU của NVIDIA
DRAM	Dynamic Random-Access Memory	Bộ nhớ truy cập ngẫu nhiên động
FFI	Foreign Function Interface	Giao diện gọi hàm giữa các ngôn ngữ
GEMM	General Matrix Multiplication	Phép nhân ma trận tổng quát
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
IR	Intermediate Representation	Biểu diễn trung gian
JIT	Just-In-Time (compilation)	Biên dịch tức thời khi chạy
LLVM	LLVM Compiler Infrastructure (ban đầu là “Low Level Virtual Machine”)	Hạ tầng trình biên dịch LLVM
REPL	Read–Eval–Print Loop	Vòng lặp đọc – thực thi – in kết quả

PHẦN I: Tổng quan dự án

I.1. Lý do chọn dự án

Sự phát triển của AI trong thời gian gần đây đã dẫn đến sự bùng nổ của nhiều thuật toán máy học và mô hình học sâu. Do vậy, nhu cầu cho năng lực tính toán để học trên những bộ dữ liệu khổng lồ cũng từ đó tăng.

Hiện nay, một trong những phép toán được sử dụng rộng rãi cho thuật toán máy học là GEMM. Trong quá trình học, mỗi phép nhân ma trận sẽ tạo ra một ma trận khác và ma trận mới sẽ được nối với những phép nhân ma trận khác. Để xử lý nhiều thao tác, tác vụ và quá trình khác nhau, những mô hình máy học cần thực hiện hàng triệu phép tính ma trận. Do những phép tính ma trận này so với các phép tính thông thường khác có tính đặc thù về thời gian rất cao, vì vậy những phép tính tiếp theo dần bị dồn lên bộ nhớ đệm và các bước trung gian cho đến khi những chương trình quản lý bộ nhớ hoặc quy tắc của máy tính giải phóng bộ nhớ.

Việc lặp lại quá trình này đối với mô hình máy học trở thành một vấn đề lớn trong việc hoạt động dưới một khối lượng công việc khổng lồ, để lại hậu quả lớn cho hiệu suất và độ trễ của mô hình trên thiết bị và tài nguyên phần cứng nó đang sử dụng.

Bởi vậy, phần mềm góp phần tối ưu hóa việc quản lý bộ nhớ xung quanh các phép toán GEMM, đảm bảo bộ đệm, dữ liệu đệm và con trỏ được giải phóng sớm nhất có thể và giảm phí tổn từ kết quả của việc dồn bộ nhớ đệm.

Dự án được lấy cảm hứng từ những nghiên cứu về hệ thống bộ nhớ đọc phá hủy, ngôn ngữ Sere [5], ngôn ngữ Clean [4] và thư viện tối ưu phép nhân ma trận ở GPU.

I.2. Câu hỏi nghiên cứu

Các chi phí phát sinh vốn có, cũng như khả năng đánh giá quá cao mức chi phí bộ nhớ hoặc tính thực tiễn của việc tối ưu bộ nhớ là một điều cần phải được cân nhắc. Do đó, dự án nghiên cứu của nhóm chúng em đưa ra các giả thuyết sau:

- Ngữ nghĩa đọc phá hủy có thể giảm mức sử dụng bộ nhớ đỉnh đối với các khối lượng công việc chứa nhiều ma trận trung gian có vòng đời ngắn.
- Đọc phá hủy có thể giảm số lượng hoặc chi phí cấp phát bộ đệm tạm thời khi các bộ đệm đã bị tiêu thụ có thể được tái sử dụng một cách an toàn.
- Với các khối lượng công việc mà việc quản lý bộ nhớ chiếm tỷ trọng đáng kể trong tổng thời gian thực thi, việc tái sử dụng bộ đệm có thể cải thiện hiệu năng đầu-cuối.

Đồng thời qua nghiên cứu này có thể trả lời được câu hỏi chính về việc “Làm thế nào để tối ưu hóa việc tính toán các phép nhân ma trận ở trong bộ nhớ đệm và trung gian để giảm chi phí máy tính?”.

I.3. Mục tiêu nghiên cứu

Mục tiêu của ngôn ngữ là xác định liệu ngữ nghĩa đọc phá hủy ở cấp độ ngôn ngữ có thể mang lại lợi ích đo lường được về quản lý bộ nhớ cho các chương trình GPU xử lý

ma trận, đồng thời vẫn giữ được hiệu năng của các kernel nhân ma trận đã được tối ưu hay không.

Dự án sẽ được triển khai và đánh giá thông qua các mục tiêu sau:

1. Định nghĩa cú pháp cốt lõi tương tự Python và ngữ nghĩa đọc phá hủy của ngôn ngữ.
2. Xây dựng bộ phân tích từ vựng (lexer), bộ phân tích cú pháp (parser), AST và quy trình sinh mã dựa trên LLVM.
3. Sinh LLVM IR cho các phép toán số học và phép toán ma trận.
4. Tích hợp phép nhân ma trận GPU đã được tối ưu thông qua cuBLAS.
5. Hiện thực cơ chế theo dõi quyền sở hữu và việc tiêu thụ một cách tường minh cho các toán hạng ma trận.
6. Xác định khi nào các bộ đệm ma trận đã bị tiêu thụ có thể được tái sử dụng an toàn.
7. Kiểm chứng kết quả sinh ra so với một tham chiếu số học độc lập.
8. Đo mức sử dụng bộ nhớ GPU đỉnh và hành vi cấp phát bộ đệm tạm thời.
9. Đo cả thời gian thực thi kernel GPU và tổng thời gian chạy đầu-cuối của chương trình.
10. So sánh việc thực thi có đọc phá hủy với một cài đặt tương đương không phá hủy.
11. So sánh ngôn ngữ với các thư viện GPU đã tối ưu làm mốc tham chiếu khi phù hợp.
12. Lưu trữ nhật ký đo hiệu năng, kết quả kiểm tra tính đúng đắn, đầu ra của trình biên dịch và các sản phẩm build để đảm bảo khả năng tái lập kết quả thực nghiệm.

I.4. Tính mới nghiên cứu

Điểm mới của dự án là liên kết trực tiếp *thư viện runtime cuBLAS* [2] đã được biên dịch sẵn cho một ứng dụng cụ thể, được đánh giá bằng một phương pháp cụ thể, theo một mô hình ràng buộc tương tự *hệ thống kiểu tuyến tính nghiêm ngặt*. Một số ngôn ngữ khác cũng đã sử dụng các ngữ nghĩa để theo dõi việc tiêu thụ tương tự: hệ thống sở hữu (ownership) của Rust [6] áp đặt nguyên tắc chỉ sử dụng một lần thông qua hệ thống kiểu affine (biến thể nói lỏng của hệ thống kiểu tuyến tính, chỉ sử dụng biến số một lần duy nhất), và Futhark [7] – một ngôn ngữ mảng thuần hàm sử dụng kiểu duy nhất (uniqueness types) để hỗ trợ cập nhật tại chỗ một cách an toàn.

Tuy nhiên, một hạn chế của Futhark trong quá trình vận hành là việc hệ thống tự sinh các kernel (nhân) GPU từ mã nguồn thay vì tích hợp với các thư viện hiệu năng cao bên ngoài được duy trì độc lập như cuBLAS, bởi vậy nó được xem là một trong những trở ngại thực tiễn của ngôn ngữ này. Ngược lại, ngôn ngữ của chúng em tách biệt quá trình phân tích tiêu thụ tại thời điểm biên dịch khỏi quá trình thực thi kernel.

PHẦN II: Cơ sở lý thuyết

Phép nhân ma trận có thể được biểu diễn như sau:

$$C_{i,j} = \sum_{k=0}^{N-1} A_{i,k} B_{k,j} \quad (1)$$

Mỗi phần tử trong ma trận kết quả C được tạo ra bằng cách nhân một hàng của A với một cột của B rồi cộng các tích lại. Các cài đặt hiệu quả phải quản lý việc di chuyển dữ liệu, tái sử dụng bộ nhớ và vòng đời của các bộ đệm toán hạng.

$$T_1 = AB, \quad T_2 = T_1 C, \quad T_3 = T_2 D, \quad (2)$$

Thực tế, nhiều bộ đệm trung gian cùng tồn tại tại một thời điểm. Vùng nhớ của các bộ đệm này nên được xóa và tái sử dụng ngay sau khi các phép toán tương ứng hoàn tất hoặc khi vòng đời của đối tượng kết thúc để có được hiệu quả tốt nhất.

Đọc phá hủy (destructive read) là thao tác đọc bộ nhớ mà bản thân nó xóa đi giá trị được đọc: dữ liệu bị hủy ngay tại thời điểm được truy cập. Nếu cần giữ lại giá trị, một thao tác ghi ngược (write-back) phải khôi phục nó ngay lập tức, hoặc một bộ đệm/bộ nhớ đệm bên ngoài phải lưu giữ một bản sao.

Các nghiên cứu liên quan về hệ thống kiểu tuyến tính cũng tiếp cận tương tự ở cấp độ ngôn ngữ, yêu cầu mỗi biến phải được sử dụng đúng một lần.

Trong lịch sử, đọc phá hủy phổ biến trong thiết kế phần cứng và mạch điện hơn là trong phần mềm. Dybdahl et al. [1] đã nghiên cứu hành vi đọc phá hủy trong DRAM nhúng và nhận thấy rằng việc loại bỏ bộ đệm ghi ngược (write-back buffer), thay vào đó dựa vào bộ nhớ đệm bằng cách đọc dữ liệu vào bộ nhớ đệm mà không khôi phục ngay về DRAM và chỉ ghi ngược khi dòng bộ nhớ đệm bị thay thế, mang lại lợi ích đo được về điện năng và hiệu năng.

Mặc dù kết quả này chỉ áp dụng cho một hệ thống bộ nhớ phần cứng cụ thể, nó gợi mở câu hỏi rộng hơn mà dự án này nghiên cứu: liệu một nguyên tắc tương tự, tức tiêu thụ khi đọc, khi được áp dụng ở cấp độ phần mềm/trình thông dịch cho các khối lượng công việc nặng về GEMM, có thể mang lại kết quả tương tự hay không?

II.1. Ngữ nghĩa đọc phá hủy

Một thao tác đọc phá hủy coi một giá trị là đã bị tiêu thụ khi một phép toán có quyền sử dụng độc quyền giá trị đó. Sau khi bị tiêu thụ, giá trị ban đầu không còn khả dụng cho các phép toán tiếp theo, trừ khi lập trình viên tạo bản sao một cách tường minh hoặc bảo toàn nó bằng cách khác.

Đối với các phép toán ma trận, điều này tạo cơ hội tái sử dụng vùng nhớ. Ví dụ,

$$\begin{aligned} C &= A @ B \\ D &= C @ W \end{aligned}$$

trong đó ký hiệu @ biểu thị phép nhân ma trận. Điều này có thể cho phép vùng nhớ của A hoặc C được tái sử dụng khi trình biên dịch xác định rằng giá trị tương ứng không còn được sử dụng ở đâu nữa.

Mục đích của cơ chế này không phải là thay đổi phép tính toán học của phép nhân ma trận. Thay vào đó, nó thay đổi vòng đời của các bộ đệm liên quan đến phép tính. Nếu một bộ đệm có thể được tái sử dụng mà không cần sao chép nội dung, cài đặt có thể giảm mức tiêu thụ bộ nhớ đỉnh và số lần cấp phát.

Tối ưu này phụ thuộc vào đặc thù của khối lượng công việc. Một ma trận còn được sử dụng ở nhiều nơi về sau thì không thể bị hủy an toàn sau lần sử dụng đầu tiên. Trong trường hợp đó, cài đặt có thể cần sao chép, mượn (borrow) hoặc dùng một cơ chế khác để bảo toàn giá trị. Do đó, ngữ nghĩa đọc phá hủy có thể tạo ra sự đánh đổi giữa hiệu quả sử dụng bộ nhớ và tính linh hoạt khi tái sử dụng giá trị.

Vì vậy, ngôn ngữ coi đọc phá hủy là một cơ chế sở hữu tường minh (bị quản lý), thay vì mặc định rằng việc tiêu thụ mọi toán hạng chắc chắn sẽ cải thiện hiệu năng.

PHẦN III: Thiết kế và phương pháp

III.1. Thiết kế cú pháp

Chúng em lựa chọn quy ước cú pháp tương tự Python cho ngôn ngữ lập trình:

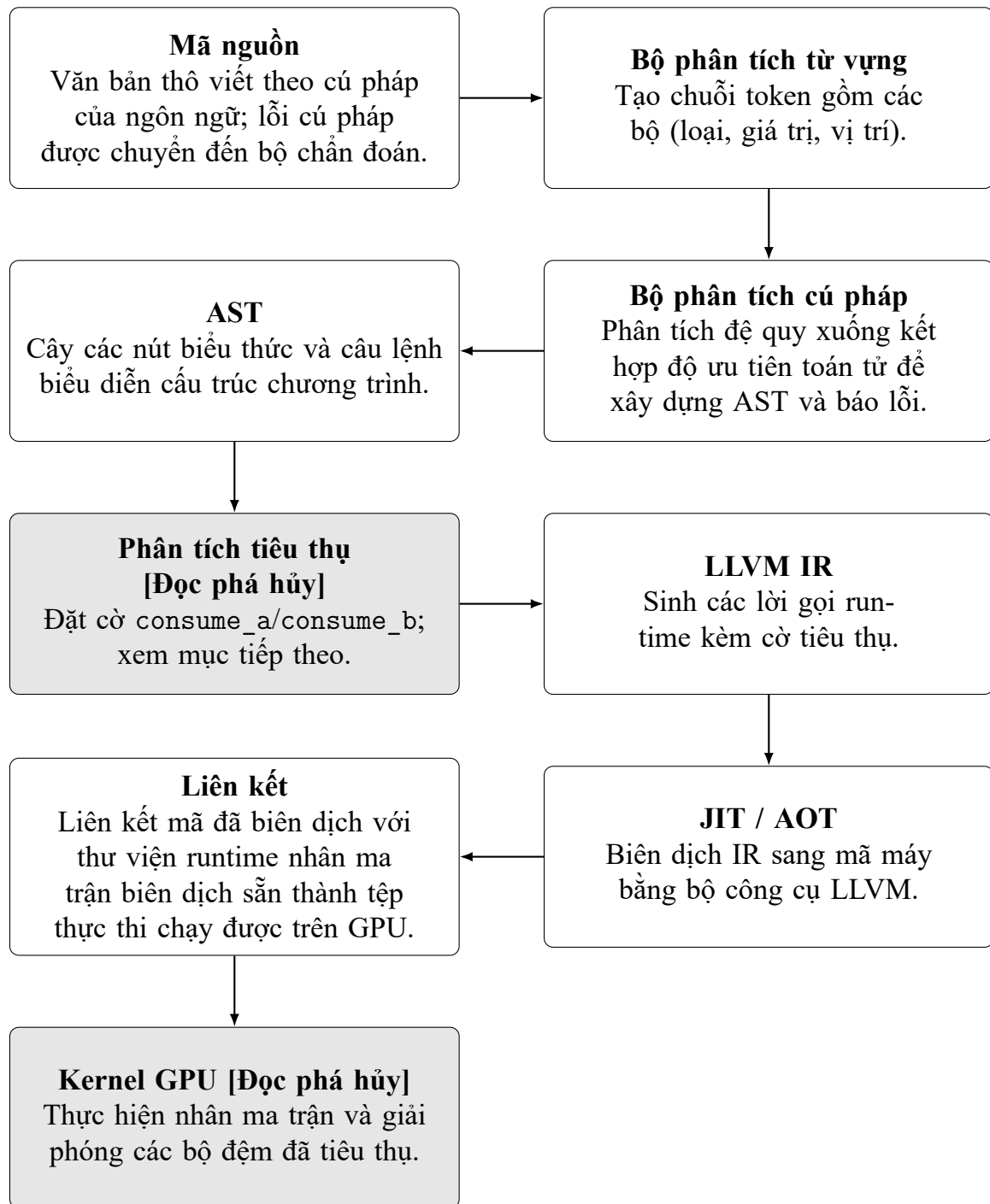
- Chú thích (comment) bằng dấu thăng (#).
- Các kiểu nguyên thủy: số nguyên (Integer), số thực (Float), luận lý (Boolean) và chuỗi (String).
- Khai báo biến.
- Định nghĩa hàm bằng từ khóa 'def'.
- Từ khóa extern cho liên kết runtime/FFI (liên ngôn ngữ).
- Độ ưu tiên toán tử.
- Cấu trúc điều khiển luồng.

Tuy nhiên, vì tính thực tiễn khi cài đặt và các quyết định thiết kế khác, chúng em đã điều chỉnh một số điểm so với cú pháp Python gốc:

- Độ ưu tiên đặc biệt cho clone() dùng để sao chép vùng nhớ.
- Sử dụng dấu ngoặc nhọn thay cho thụt lề để xác định phạm vi của hàm.

III.2. Quy trình xử lý

Toàn bộ quy trình xử lý của ngôn ngữ, từ mã nguồn đến thực thi trên GPU, được mô tả trong Hình 1. Quá trình sinh mã được xây dựng trên hạ tầng trình biên dịch LLVM [3].



Hình 1. Quy trình biên dịch của ngôn ngữ. Các khối tô xám biểu thị những bước liên quan đến ngữ nghĩa đọc phá hủy.

III.3. Cơ chế giải phóng khi tiêu thụ và runtime cuBLAS

Cơ chế được đề cập trong [1] sẽ được áp dụng với triết lý thiết kế đã điều chỉnh để hoạt động ở cấp độ phần mềm; các lựa chọn thiết kế được tóm tắt trong Bảng 1.

Đối với việc liên kết cuBLAS đã biên dịch sẵn:

- cuBLAS cần được liên kết động.
- Chương trình điều khiển (driver) cần liên kết và được phân phối kèm các tệp nh

phần đã biên dịch của cuBLAS.

- Để thuận tiện, chương trình được phân phối dưới dạng một tệp đóng gói.

Bảng 1. Các lựa chọn hiện thực cơ chế trong thiết kế.

Cơ chế	Hiện thực
Dữ liệu trước tiên được đọc từ DRAM vào bộ nhớ đệm.	Không có khác biệt.
Dữ liệu được ghi ngược về DRAM khi dòng bộ nhớ đệm bị thay thế, bất kể dữ liệu có bị thay đổi hay không.	Các toán hạng được giải phóng vô điều kiện với giả định rằng bộ nhớ đệm luôn bị thay đổi.
Tái sử dụng kiến trúc bộ nhớ đệm sẵn có.	Sử dụng cơ chế giải phóng khi thoát phạm vi (scope-exit) để quản lý bộ nhớ đệm.
Che giấu độ trễ ghi ngược bằng cách chồng lấp nhiều bank (bank interleaving - kỹ thuật phân phối yêu cầu truy cập bộ nhớ luân phiên nhiều bank thay vì một bank duy nhất).	Ẩn chi phí giải phóng bộ nhớ phía sau các lần khởi chạy kernel GPU.

PHẦN IV: Quy trình, chế tạo và kiểm thử

IV.1. Quy trình thực hiện

Phần front-end của trình biên dịch được viết bằng C++. Mỗi giai đoạn được thiết kế trước, sau đó lập trình, rồi gỡ lỗi độc lập để phát hiện lỗi, và cuối cùng các giai đoạn được ghép nối theo dạng mô-đun. Các tệp mã nguồn chính được liệt kê trong Bảng 2.

Bảng 2. Các tệp mã nguồn chính

Tệp	Chức năng
Lexer.h/.cpp	Tách mã nguồn của ngôn ngữ thành token
Parser.h/.cpp	Xây dựng AST
ExprAST.h	Định nghĩa các nút AST
CodeGen.cpp	Sinh LLVM IR
CuBLASRuntime.cpp	Liên kết cuBLAS cho phép nhân ma trận
main.cpp	Chương trình điều khiển / REPL

Bảng 3. Kết quả kiểm tra tính đúng đắn

Trường hợp kiểm tra	Kết quả
Nhân ma trận $N \times N$ so với tham chiếu (thông thường)	Chưa có
Phát hiện lỗi tái sử dụng sau đọc phá hủy (khi chạy)	Chưa có
Trường hợp biên (rỗng, 1x1, không vuông)	Chưa có

IV.2. Kiểm thử

- Nhân ma trận $N \times N$ so với tham chiếu (thông thường): đánh giá một chuỗi phép nhân ma trận nhằm kiểm tra khả năng chịu tải và đánh giá mức cải thiện đầu-cuối.
- Phát hiện lỗi tái sử dụng sau đọc phá hủy: kiểm tra tại cả thời điểm chạy và thời điểm biên dịch để phát hiện lỗi.
- Trường hợp biên: đánh giá các trường hợp biên nhằm kiểm tra tính tương thích và độ chặt chẽ của thuật toán.

IV.3. Đánh giá hiệu năng

Việc đánh giá hiệu năng được thiết kế để phân biệt cải thiện về quản lý bộ nhớ với cải thiện của bản thân phép nhân ma trận.

Ba cài đặt chính sẽ được so sánh:

1. Một cài đặt sử dụng thư viện GPU đã tối ưu, làm mốc tham chiếu cho hiệu năng kernel.
2. Ngôn ngữ với ngữ nghĩa giá trị thông thường (không phá hủy).
3. Ngôn ngữ với ngữ nghĩa đọc phá hủy.

Khi phù hợp, một cài đặt C++ hoặc CUDA đơn giản cũng sẽ được đưa vào làm mốc cơ sở.

Phép so sánh quan trọng nhất là giữa hai cấu hình của ngôn ngữ. Cả hai cấu hình thực hiện cùng các phép toán và sử dụng cùng một cài đặt nhân ma trận GPU bên dưới. Phép so sánh này tách biệt tác động của ngữ nghĩa đọc phá hủy hiệu quả hơn so với việc so sánh với một cài đặt không liên quan.

Bộ đánh giá sẽ bao gồm cả các phép nhân ma trận riêng lẻ lẫn các chuỗi phép toán ma trận. Kích thước ma trận gồm nhỏ, trung bình, lớn và không vuông. Điều này cho phép xác định liệu tác động của đọc phá hủy có phụ thuộc vào quy mô khối lượng công việc hay không.

Các số liệu sau sẽ được thu thập:

- Thời gian thực thi kernel GPU.
- Thời gian thực thi đầu-cuối.

- Mức tiêu thụ bộ nhớ GPU đỉnh.
- Số lần cấp phát bộ đệm tạm thời.
- Số lần tái sử dụng bộ đệm.
- Thời gian truyền dữ liệu từ máy chủ sang thiết bị và từ thiết bị về máy chủ (nếu có).
- Chi phí của trình biên dịch và môi trường thực thi.
- Sai số số học so với cài đặt tham chiếu.
- Công suất tiêu thụ của GPU.

Các phép đo trên GPU sẽ được lặp lại nhiều lần với số vòng khởi động (warm-up) phù hợp. Chi phí khởi tạo CUDA [9] và chi phí biên dịch sẽ được tách riêng khỏi các phép đo ở trạng thái ổn định khi có thể.

Kết quả đánh giá sẽ báo cáo cả giá trị tuyệt đối lẫn mức thay đổi tương đối. Không có cải thiện hiệu năng nào được quy cho ngữ nghĩa đọc phá hủy trừ khi các số liệu đo được ủng hộ cách diễn giải đó. Việc giảm mức sử dụng bộ nhớ đỉnh mà không giảm thời gian chạy vẫn được xem là một kết quả có ý nghĩa.

IV.4. Môi quan hệ kỳ vọng giữa ngôn ngữ và hiệu năng nhân ma trận

Có thể kỳ vọng một ngôn ngữ với ngữ nghĩa đọc phá hủy sẽ mang lại lợi ích về quản lý bộ nhớ. Các phép toán số học như phép nhân ma trận được giao cho kernel cuBLAS đã tối ưu, vì vậy việc cài đặt được kỳ vọng không ảnh hưởng đến hiệu năng tính toán so với cài đặt trên các ngôn ngữ khác. Tuy nhiên, hiệu năng đầu-cuối được kỳ vọng sẽ cải thiện nhờ các tối ưu của ngôn ngữ, bao gồm:

- Giảm cấp phát tạm thời
- Giảm vòng đời bộ đệm
- Giảm áp lực lên bộ nhớ GPU

Về lý thuyết, lợi ích tăng theo số lượng phép toán nối tiếp và tensor trung gian: một phép nhân ma trận riêng lẻ hầu như không được hưởng lợi, trong khi các chuỗi phép toán dài với nhiều giá trị trung gian có vòng đời ngắn được hưởng lợi nhiều nhất.

Thời gian thực thi kỳ vọng có thể được phân tách gần đúng như sau:

$$T_{\text{total}} = T_{\text{language}} + T_{\text{memory}} + T_{\text{launch}} + T_{\text{kernel}}. \quad (3)$$

trong đó T_{language} là chi phí của ngôn ngữ, T_{memory} là chi phí quản lý bộ nhớ, T_{launch} là chi phí khởi chạy kernel và T_{kernel} là thời gian thực thi kernel.

Ngữ nghĩa đọc phá hủy chủ yếu nhắm vào T_{memory} . Chúng không được kỳ vọng sẽ giảm đáng kể T_{kernel} khi cùng một kernel GPU đã tối ưu được sử dụng.

Với một phép nhân ma trận lớn, T_{kernel} có thể chiếm phần lớn tổng thời gian thực thi, dẫn đến cải thiện đầu-cuối khó quan sát được. Tuy nhiên, với các khối lượng công việc

chứa nhiều tensor trung gian hoặc các phép toán ma trận nhỏ, T_{memory} và các chi phí khác của môi trường thực thi có thể trở nên đáng kể hơn.

Do đó, việc đánh giá hiệu quả của ngôn ngữ chủ yếu dựa vào phép đo T_{memory} .

IV.5. Minh chứng và khả năng tái lập

Ở giai đoạn hiện tại, minh chứng chỉ giới hạn ở phần front-end đã được hiện thực (bộ phân tích từ vựng, bộ phân tích cú pháp, AST và chương trình điều khiển). Kết quả cho việc thực thi GEMM và tích hợp cuBLAS sẽ được bổ sung khi các giai đoạn đó được hiện thực. Ví dụ biên dịch với phần front-end hiện tại:

```
1 def basicadd(int i, int k) {  
2     return i + k;  
3 }  
4 # Basic compilation of source code, addition only. No showcase of  
   GEMMs or cuBLAS integration.  
5
```

Đoạn mã 1. Ví dụ biên dịch

PHẦN V: Kết luận

V.1. Tiến độ

Dự án đã hoàn thành một phần khối lượng công việc dự kiến, bao gồm bộ phân tích từ vựng, bộ phân tích cú pháp, AST và một chương trình điều khiển (driver) cơ bản. Thiết kế cho phần còn lại của quy trình đã được hoàn thiện, nhưng việc hiện thực các tính năng ở các giai đoạn sau của quy trình chưa được bắt đầu do hạn chế về thời gian.

V.2. Hướng phát triển

Các tính năng ở các giai đoạn sau của quy trình, bao gồm việc tích hợp thư viện runtime nhân ma trận đã biên dịch sẵn, vẫn cần được hiện thực. Các tính năng hiện đang được lên kế hoạch gồm:

- Kiểu vector và ma trận
- Chương trình điều khiển chẩn đoán và bộ chẩn đoán (diagnostic engine)
- Phân tích tiêu thụ tĩnh (kiểm tra đọc phá hủy) và xử lý các bộ đệm cũng như giá trị trung gian của kernel GPU
- Liên kết với runtime nhân ma trận đã biên dịch sẵn để tạo ra tệp thực thi có khả năng khởi chạy kernel GPU
- Tách quá trình sinh mã khỏi AST để việc hạ cấp mã (lowering) rõ ràng hơn
- Các lượt tối ưu (optimization passes)
- Hệ thống macro (khai báo liên kết, trừu tượng hóa mẫu chuỗi nhân ma trận)

V.3. Ý nghĩa

Dự án này vừa là một thử nghiệm nhằm khám phá các cải tiến tiềm năng cho các phép toán GEMM trong các thuật toán học máy và AI, bổ sung cho thư viện cuBLAS [2] vốn đã được tối ưu rất cao, vừa là một phản hồi trước một vấn đề rộng hơn của lĩnh vực. Quá trình huấn luyện mô hình hiện nay tiêu tốn lượng bộ nhớ khổng lồ, và nhu cầu từ hạ tầng AI đã vượt quá nguồn cung chip nhớ toàn cầu, góp phần khiến giá DRAM và NAND tăng liên tục, gây áp lực tài chính lên các doanh nghiệp công nghệ thông tin, các viện nghiên cứu khoa học và cả người tiêu dùng [8]. Các thuật toán tính toán cho phép toán GEMM đã được tối ưu *gần đến giới hạn lý thuyết* về thông lượng; thay vào đó, nghiên cứu này nhằm đến hiệu quả sử dụng bộ nhớ, sử dụng một hệ thống sở hữu mới để giảm dung lượng bộ nhớ cần thiết cho mỗi khối lượng công việc, như một đóng góp nhằm giảm bớt áp lực từ phía cầu đang thúc đẩy tình trạng thiếu hụt bộ nhớ hiện nay.

Về mặt thực tiễn, việc tái sử dụng các bộ đệm tensor trung gian và giảm chi phí cấp phát khi chạy có thể hỗ trợ việc huấn luyện và suy luận các mô hình học máy nhanh hơn.

TÀI LIỆU THAM KHẢO

- [1] Haakon Dybdahl, Per Gunnar Kjeldsberg, Marius Grannæs, and Lasse Natvig, “Destructive-read in embedded DRAM, impact on power consumption,” *Journal of Embedded Computing*, vol. 2, no. 1, pp. 83–96, 2006.
- [2] NVIDIA, “cuBLAS Library.” <https://developer.nvidia.com/cublas>
- [3] LLVM Project, “The LLVM Compiler Infrastructure.” <https://llvm.org>
- [4] Clean Language Team, “The Clean Programming Language.” <https://clean.cs.ru.nl>
- [5] Sere Language Project, “Sere Programming Language.” <https://clean-lang.org>
- [6] N. D. Matsakis and F. S. Klock, “The Rust language,” in *Proc. ACM SIGAda Ada Letters*, 2014. <https://www.rust-lang.org>
- [7] T. Henriksen, N. G. W. Serup, M. Elsmann, F. Henglein, and C. E. Oancea, “Futhark: purely functional GPU-programming with nested parallelism and in-place array updates,” in *Proc. 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017. <https://futhark-lang.org>
- [8] TrendForce, “Memory Price Outlook for 1Q26 Sharply Upgraded,” <https://www.trendforce.com/presscenter/news/20260202-12911.html>
- [9] NVIDIA, “CUDA Toolkit.” <https://developer.nvidia.com/cuda/toolkit>